

Курсова робота з ОП — варіант №10

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»**

Кафедра обчислювальної техніки

КУРСОВА РОБОТА

з дисципліни «Основи програмування»

на тему:

**«Сайт онлайн-новин. Додавання новин здійснюється
зареєстрованими авторами» (варіант №10)**

Виконав:

студент 1 курсу, групи _____

Ступак Захар Іванович

Керівник:

—

Робота захищена з оцінкою:

«___» _____

Київ — 2026

РЕФЕРАТ

Пояснювальна записка: 30 с., 1 рис., 5 табл., 6 джерел, 1 додаток.

Об'єктом розробки є веб-застосунок для публікації онлайн-новин зареєстрованими авторами. Предмет розробки — об'єктно-орієнтована модель предметної області мовою C++ та реалізація HTTP-інтерфейсу до неї як окремого виконуваного файлу.

Мета роботи — спроектувати ієрархію класів для опису сутностей «автор», «рубрика», «тег» і «новина», запровадити операції додавання, заміни, видалення, пошуку, фільтрації та сортування новин і надати користувачам доступ до цих операцій через HTTP-інтерфейс.

Програма реалізована мовою C++17. Для HTTP-протоколу використано сторонню заголовкову бібліотеку `cpp-httplib` (v0.18.1), яка підтягується через `CMake FetchContent`. Дані зберігаються у файлі простого текстового формату.

Ключові слова: C++17, ООП, наслідування, поліморфізм, HTTP, `cpp-httplib`, `CMake`, серверний рендеринг, сесії, онлайн-новини.

ЗМІСТ

РЕФЕРАТ	2
ЗМІСТ	3
ВСТУП.....	5
1. ПОСТАНОВКА ЗАДАЧІ	6
1.1. Формулювання.....	6
1.2. Функціональні вимоги	6
1.3. Нефункціональні вимоги	6
2. АНАЛІЗ ТА ВИБІР ЗАСОБІВ РЕАЛІЗАЦІЇ.....	8
2.1. Мова програмування	8
2.2. HTTP-бібліотека	8
2.3. Збірка	8
2.4. Сховище даних.....	8
2.5. Захист пароля	8
3. ОБ'ЄКТНО-ОРІЄНТОВАНА МОДЕЛЬ	10
3.1. Огляд.....	10
3.2. Клас DateTime	10
3.3. Абстрактний клас User.....	11
3.4. Клас Author.....	11
3.5. Класи Category та Tag	11
3.6. Клас News.....	11
3.7. Клас SearchCriteria.....	11
3.8. Клас NewsRepository	12
3.9. Принципи ООП у коді.....	12
4. ОПИС ПРОГРАМИ	13
4.1. Структура проекту.....	13
4.2. Збірка	14
4.3. HTTP-маршрути.....	14
4.4. Реалізація пошуку, фільтрації та сортування	14
4.5. Автентифікація і сесії.....	15
4.6. Серверний рендер HTML.....	15
4.7. Збереження даних	15
5. ІНСТРУКЦІЯ КОРИСТУВАЧА.....	16
5.1. Системні вимоги	16

5.2. Збірка	16
5.3. Параметри командного рядка.....	16
5.4. Робота у браузері	16
5.5. Завершення роботи.....	17
ВИСНОВКИ	18
ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ	19
ДОДАТОК А. ЛІСТИНГ КЛЮЧОВИХ ЗАГОЛОВКІВ	20

ВСТУП

Онлайн-новини є одним з найпоширеніших сервісів сучасного Інтернету. У більшості випадків такі сервіси будуються за схемою «автор-редакція-читач»: зареєстрований автор пише матеріал, відносить його до певної рубрики й супроводжує тегами, а читач переглядає стрічку, фільтруючи її за датою, тематикою чи автором. Для курсової роботи з дисципліни «Основи програмування» (варіант №10) поставлено задачу спроектувати і реалізувати таку інформаційну систему.

За вимогами методичних рекомендацій програма повинна підтримувати додавання, видалення, заміну, пошук, сортування та фільтрацію новин за різними критеріями (назвою, тематикою, рубрикою, автором, періодом). Окремо вимагається, щоб об'єктно-орієнтована модель була оформлена як самостійний модуль (бібліотека), а взаємодія з користувачем — як інший проект, що до неї звертається. Прямого використання консольного вводу/виводу слід уникати.

Тема варіанту №10 — «Сайт. Онлайн-новини» — прямо вказує на веб-інтерфейс. Тому в роботі розроблено повноцінний HTTP-сервер мовою C++17 на основі сторонньої заголовкової бібліотеки `cpp-httplib`. Сервер відрендерює HTML-сторінки на стороні сервера, обробляє форми та керує сесіями зареєстрованих авторів. Це дозволило одночасно виконати вимогу про мінімізацію консолі (вся взаємодія йде через браузер) і вимогу про «сайт» з умови задачі.

Перший розділ роботи містить формальну постановку задачі і перелік функціональних вимог. У другому розділі обґрунтовано вибір мови, бібліотек і системи збірки. Третій розділ присвячено об'єктно-орієнтованій моделі: для кожного класу описано призначення, поля та основні методи, наведено UML-діаграму. У четвертому розділі описано архітектуру серверної частини та реалізацію конкретних функціональних вимог. П'ятий розділ — інструкція з встановлення, збірки й користування програмою. У висновках підбито підсумки та зазначено можливі напрямки розвитку.

1. ПОСТАНОВКА ЗАДАЧІ

1.1. Формулювання

Розробити програмний засіб, який реалізує сайт онлайн-новин. У системі діють зареєстровані автори, що можуть публікувати власні матеріали, та читачі (анонімні відвідувачі), що мають доступ до читання, пошуку та фільтрації стрічки. Кожна новина належить до однієї рубрики (категорії) і може мати довільну кількість тегів.

1.2. Функціональні вимоги

Програма повинна забезпечувати такі операції:

- реєстрацію та автентифікацію автора;
- додавання нової новини зареєстрованим автором — з обов'язковими полями «заголовок», «текст», «рубрика» та довільним списком тегів;
- заміну (редагування) власної новини автора;
- видалення власної новини автора;
- перегляд стрічки усіх новин з можливістю розгорнути окрему публікацію;
- пошук новин за фрагментом заголовка;
- фільтрацію за рубрикою, тегом, автором та діапазоном дат;
- сортування результату за датою публікації, заголовком чи іменем автора, у двох напрямках;
- збереження даних між запусками програми (персистентне сховище).

1.3. Нефункціональні вимоги

- реалізація мовою C++ (за варіантом — C++ або C#);
- використання об'єктно-орієнтованого підходу;
- «інтерфейс» (взаємодія з користувачем) — окремий проект, що звертається до бібліотеки предметної області;
- мінімум консольного вводу/виводу — взаємодія з користувачем через веб-браузер;

- паролі зберігаються у хешованому вигляді з сіллю;
- робота під macOS, Linux або Windows.

2. АНАЛІЗ ТА ВИБІР ЗАСОБІВ РЕАЛІЗАЦІЇ

2.1. Мова програмування

Обрано C++17. Стандарт дає `std::optional`, `std::variant`, `structured bindings`, які зручні у роботі з пошуком та фільтрацією, а також `std::unique_ptr` для керування часом життя об'єктів. C++17 підтримують усі поширені компілятори: Apple Clang, GCC, MSVC.

2.2. HTTP-бібліотека

Для веб-сервера використано `cpp-httplib v0.18.1` — заголовкову бібліотеку HTTP/1.1 (ліцензія MIT). Вона бере на себе парсинг запитів, формування відповідей, роботу з куками та пул потоків, що дозволяє у коді працювати лише з прикладною логікою.

2.3. Збірка

Проект зібрано через CMake (мінімальна версія 3.16). Кореневий `CMakeLists.txt` підключає дві підпапки — `domain/` (статична бібліотека) і `server/` (виконуваний файл). `cpp-httplib` завантажується через `FetchContent`: при першій конфігурації CMake сам клонує потрібний тег із GitHub.

2.4. Сховище даних

Дані зберігаються у текстовому файлі `news.db` простого формату: окремий префікс на тип запису (C — категорія, T — тег, A — автор, N — новина), поля розділені табуляцією. Зчитування й запис інкапсульовано у методах `NewsRepository::loadFromFile()` та `saveToFile()`.

2.5. Захист пароля

Паролі у відкритому вигляді в файлі бази не зберігаються. До пароля додається випадкова 16-символьна сіль, далі обчислюється хеш через `std::hash<std::string>`, і у файл записується рядок «сіль\$хеш». При вході вводиться той самий пароль, до нього додається збережена сіль, обчислюється хеш і порівнюється з тим, що збережено. Для навчальної

задачі такого захисту достатньо; у промислових системах застосовуються спеціалізовані алгоритми типу bcrypt чи argon2.

3.3. Абстрактний клас User

Базовий клас для користувачів. Зберігає `id`, ім'я, `email`, хеш пароля й дату реєстрації. Метод `role()` — чисто віртуальний, тому `User` не можна створити напряду. Метод `canPublish()` за замовчуванням повертає `false`.

3.4. Клас Author

Нашадок `User`. Перевизначає `role()` (повертає «author») і `canPublish()` (повертає `true`). Додає поле `bio_` — короткий опис автора.

3.5. Класи Category та Tag

Категорія (рубрика) має `id`, назву та текстовий опис; тег — лише `id` та назву. Обидва класи реалізують метод `slug()` — повертають рядок-ідентифікатор, придатний для URL: українські букви транслітеруються у латинські, всі неприпустимі символи замінюються тире, регістр приводиться до нижнього. Це дозволяє формувати посилання вигляду `/category/sport`, `/tag/ai` тощо.

3.6. Клас News

Центральна сутність системи. Зберігає `id`, заголовок, повний текст, ідентифікатори автора, рубрики, список ідентифікаторів тегів (M:N зв'язок), а також момент публікації. Метод `excerpt(maxChars)` повертає короткий уривок тексту із врахуванням UTF-8 (обрізає не посередині багатобайтового символу). Метод `setTags()` переписує список тегів, видаляючи дублікати.

3.7. Клас SearchCriteria

Контейнер для опціональних умов пошуку. Кожне поле обернене у `std::optional`, тому фільтр «активний» тільки коли користувач задав це значення. Метод `matches(News)` повертає `true`, якщо запис відповідає всім заданим умовам одночасно. Сюди ж входять два `enum`-и — `SortBy` (`Date` / `Title` / `Author`) та `SortDirection` (`Ascending` / `Descending`), що використовуються при виклику `NewsRepository::search()`.

3.8. Клас NewsRepository

Кореневий агрегат предметної області. Містить чотири внутрішні колекції — авторів, категорій, тегів і новин. Усі публічні методи захищені `std::mutex`, щоб одночасний доступ із пулу потоків HTTP-сервера не пошкодив стан. Серед основних методів:

- `registerAuthor()` — створює нового користувача-автора з перевіркою унікальності `username` і `email`;
- `addNews()` / `updateNews()` / `deleteNews()` — реалізують додавання, заміну та видалення матеріалу з перевіркою посилальної цілісності (автор та рубрика повинні існувати);
- `search(criteria, sortBy, direction)` — повертає вектор знайдених новин, відфільтрованих за `SearchCriteria` і відсортованих у вказаному порядку;
- `loadFromFile()` / `saveToFile()` — серіалізують репозиторій у текстовий файл і завантажують його назад при запуску.

Commented [ZS1]: Fix list

3.9. Принципи ООП у коді

- Інкапсуляція — поля класів закриті, доступ через геттери і сеттери.
- Наслідування — `Author` успадковує `User`.
- Поліморфізм — метод `role()` оголошено віртуальним у `User` і перевизначено в `Author`.
- Абстракція — `User` містить чисто віртуальну функцію, тож його не можна створити безпосередньо.

4. ОПИС ПРОГРАМИ

4.1. Структура проєкту

Дерево каталогів програми наведено нижче. Жоден файл у domain/ нічого не знає про HTTP; реалізація мережевої частини зосереджена в server/.

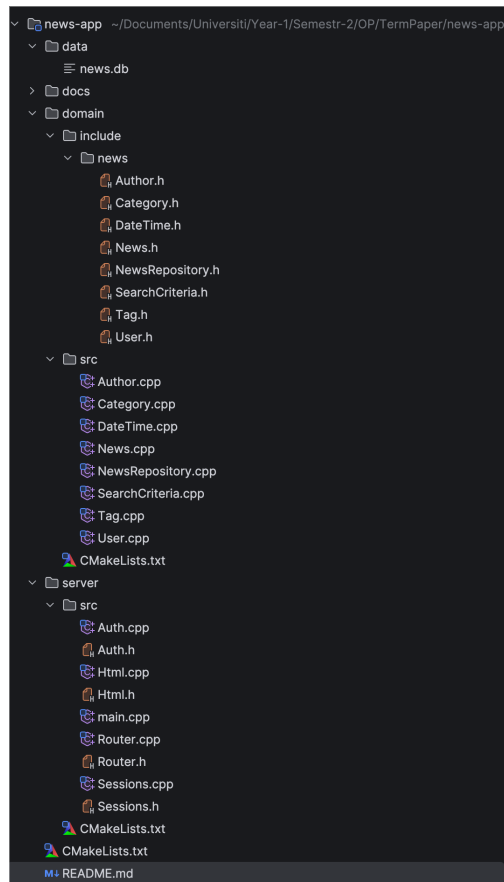


Рисунок 4.1 — Структура Проєкту

4.2. Збірка

Кореневий CMakeLists.txt задає C++17, оголошує дві FetchContent-залежності (cpp-httplib та nlohmann/json) і додає підпроекти domain і server. У підпроекті server задано окремий таргет run, який запускає скомпільований бінарник з кореня репозиторію без потреби писати довгий шлях.

4.3. HTTP-маршрути

Усі маршрути зареєстровано у функції registerRoutes() з модулю Router.cpp. Нижче — повний список (таблиця 4.1).

Таблиця 4.1 — Карта HTTP-маршрутів

Метод	Шлях	Дія
GET	/	Головна стрічка з фільтрами
GET	/news/{id}	Перегляд однієї новини
GET	/news/new	Форма створення новини
POST	/news	Створення новини (тільки автор)
GET	/news/{id}/edit	Форма редагування
POST	/news/{id}	Збереження редагованої новини
POST	/news/{id}/delete	Видалення (тільки власну)
GET	/categories, /tags, /authors	Списки сутностей
GET	/category/{slug}	Стрічка однієї рубрики
GET	/tag/{slug}	Стрічка з тегом
GET	/author/{username}	Стрічка одного автора
GET/POST	/register, /login	Ресстрація і вхід
POST	/logout	Завершення сесії

4.4. Реалізація пошуку, фільтрації та сортування

На головній сторінці (GET /) приймаються такі параметри запиту: q — фрагмент заголовку; category — id рубрики; tag — id тега; author — id автора; from / to — межі діапазону дат; sort — критерій сортування (date /

title / author), dir — напрям (asc / desc). Усі вони збираються у структуру SearchCriteria, після чого викликається NewsRepository::search(), повертаючи готовий, упорядкований вектор. На рендері форма пам'ятає значення зі строки запиту, тому при оновленні сторінки фільтр залишається.

4.5. Автентифікація і сесії

Реєстрація (POST /register) викликає NewsRepository::registerAuthor(). Пароль перед записом проходить через hashPassword() з модуля Auth.cpp — у файлі бази опиняється рядок «сіль\$хеш». При вході (POST /login) verifyPassword() порівнює введений пароль із збереженим. У разі успіху SessionStore::create(authorId) повертає випадковий токен (32 шістнадцяткові символи), який встановлюється у cookie з ім'ям session, атрибутом HttpOnly та часом життя 7 днів. На кожному наступному запиті SessionStore::authorId(token) повертає id автора, якщо сесія знайдена; інакше користувач вважається анонімним.

4.6. Серверний рендер HTML

Сторінки формуються через std::ostringstream у функціях-обробниках. Усі дані користувача проходять через escapeHtml() — це захищає від XSS. Загальний шаблон сторінки (head, header, навігація) винесено у renderLayout(); кожен обробник передає туди заголовок і вже сформований основний блок.

4.7. Збереження даних

Файл news.db має таку структуру: рядок з версією формату, рядок з лічильниками наступних id, далі — послідовність рядків C (категорія), T (тег), A (автор), N (новина). Поля розділені табуляцією, спеціальні символи екрануються (\n → \n, \t → \t, \\ → \\). Зчитування здійснюється у методі loadFromFile(), зберігання — у saveToFile(), яке викликається після кожної операції зміни (додавання/редагування/видалення новини, реєстрації автора).

5. ІНСТРУКЦІЯ КОРИСТУВАЧА

5.1. Системні вимоги

- операційна система: macOS 12+, Linux (Ubuntu 22+) або Windows 10/11;
- компілятор з підтримкою C++17 (Apple Clang, GCC $\geq$ 8, MSVC 2019+);
- CMake версії 3.16 або новішої;
- Git (потрібен лише при першому запуску CMake — для завантаження cpp-httplib);
- браузер (Chrome / Firefox / Safari) для роботи з інтерфейсом.

5.2. Збірка

У терміналі з кореня репозиторію виконати:

```
$ cmake -S . -B build  
$ cmake --build build -j
```

Після успішної компіляції виконуваний файл буде розташовано у каталозі build/bin/. Запуск:

```
$ ./build/bin/news_server
```

Альтернативно — зручний CMake-таргет:

```
$ cmake --build build --target run
```

5.3. Параметри командного рядка

Програма приймає три необов'язкові опції:

- --host HOST — інтерфейс прив'язки (за замовчуванням 127.0.0.1);
- --port PORT — TCP-порт (за замовчуванням 8080);
- --data FILE — шлях до файлу бази (за замовчуванням data/news.db).

5.4. Робота у браузері

Після запуску сервер виведе в консоль повідомлення «Сервер слухає на http://127.0.0.1:8080». При першому запуску, якщо файл бази ще не існує, програма автоматично заповнить його демонстраційними даними: трьома рубриками (Спорт, Технології, Світ), чотирма тегами та трьома новинами

зразка травня 2026 року. Доступний демонстраційний обліковий запис — логін demo, пароль demo123.

На головній сторінці розташована стрічка усіх новин з панеллю фільтрів зверху. Поля «Пошук», «Рубрика», «Тег», «З», «По» та «Сортування» комбінуються довільним чином. Після зміни значень слід натиснути «Застосувати». При натисканні на заголовок новини відкривається її повна сторінка. Для авторизації використовуються посилання «Вхід» та «Реєстрація» у правому верхньому куті. Зареєстрований користувач отримує доступ до пункту меню «+ Нова новина» та до кнопок «Редагувати» / «Видалити» на своїх власних публікаціях.

5.5. Завершення роботи

Сервер коректно зупиняється комбінацією Ctrl+C — обробник сигналу SIGINT викликає `httplib::Server::stop()`, завершуючи всі активні з'єднання й повертаючи керування процесу. База даних збережена після кожної транзакції, тож втрат не буде.

ВИСНОВКИ

У роботі реалізовано сайт онлайн-новин за варіантом №10. Виконано всі функціональні вимоги: реєстрація і вхід авторів, додавання, редагування і видалення новин, пошук за фрагментом заголовка, фільтри за рубрикою, тегом, автором і діапазоном дат, сортування за датою, заголовком чи іменем автора.

В об'єктно-орієнтованій моделі використано наслідування (Author від User), поліморфізм (віртуальний метод `role()`), абстракцію (User не можна створити безпосередньо) та інкапсуляцію (закриті поля, доступ через методи).

Код розділено на два проекти: статична бібліотека `news_domain` і виконуваний файл `news_server`, що до неї лінкується.

Можливі напрями подальшого розвитку: коментарі читачів, перехід на SQLite, JSON-API, повнотекстовий пошук, нові ролі користувачів (модератор, головний редактор).

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Stroustrup B. The C++ Programming Language. 4-те видання. Boston: Addison-Wesley, 2013. 1376 с.
2. ISO/IEC 14882:2017. Programming languages — C++. Geneva: ISO, 2017.
3. Hirose Y. cpp-http lib: A C++ header-only HTTP/HTTPS library. Версія v0.18.1. URL: <https://github.com/yhirose/cpp-http lib> (дата звернення 15.05.2026).
4. cppreference.com — довідник стандартної бібліотеки C++. URL: <https://en.cppreference.com> (дата звернення 15.05.2026).
5. CMake Documentation. Kitware. URL: <https://cmake.org/documentation/> (дата звернення 15.05.2026).

ДОДАТОК А. ЛІСТИНГ КЛЮЧОВИХ ЗАГОЛОВКІВ

Файл *domain/include/news/User.h*

```
class User {
protected:
    int      id_;
    std::string username_;
    std::string email_;
    std::string passwordHash_;
    DateTime registeredAt_;
public:
    virtual ~User() = default;
    virtual std::string role() const = 0;          // чисто віртуальний
    virtual bool      canPublish() const { return false; }
    virtual std::string display() const;
    int id() const { return id_; }
    const std::string& username() const { return username_; }
    // ...
};
```

Файл *domain/include/news/Author.h*

```
class Author : public User {
    std::string bio_;
public:
    Author(int id, std::string username, std::string email,
           std::string passwordHash, DateTime registered,
           std::string bio);
    std::string role() const override { return "author"; }
    bool      canPublish() const override { return true; }
    const std::string& bio() const { return bio_; }
    void setBio(std::string s) { bio_ = std::move(s); }
};
```

Файл *domain/include/news/SearchCriteria.h*

```
enum class SortBy      { Date, Title, Category, Author };
enum class SortDirection { Asc, Desc };

struct SearchCriteria {
    std::optional<std::string> titleContains;
    std::optional<int>         categoryId;
    std::optional<int>         tagId;
    std::optional<int>         authorId;
    std::optional<DateTime>    from;
    std::optional<DateTime>    to;
    bool matches(const News& n) const;
```

```
bool empty() const;
};
```

Файл *server/src/Auth.cpp* (фрагмент)

```
std::string hashPassword(const std::string& password) {
    std::string salt = randomHex(16);
    auto h = std::hash<std::string>{}(salt + password);
    return salt + "$" + toHex(static_cast<unsigned long long>(h));
}

bool verifyPassword(const std::string& password,
                    const std::string& stored) {
    auto pos = stored.find('$');
    if (pos == std::string::npos) return false;
    std::string salt = stored.substr(0, pos);
    std::string hash = stored.substr(pos + 1);
    auto h = std::hash<std::string>{}(salt + password);
    return toHex(static_cast<unsigned long long>(h)) == hash;
}
```

Файл *server/src/main.cpp* (фрагмент)

```
news::NewsRepository repo;
repo.loadFromFile(dataFile);
seedIfEmpty(repo);
repo.saveToFile(dataFile);

news_server::SessionStore sessions;
httplib::Server server;
news_server::registerRoutes(server, repo, sessions, dataFile);
server.listen(host, port);
```